AXIOM: Correctness-Aware Multi-Agent Slice Admission Control with Federated Training

Manoj Prasad Kunasegran[1], Wai Leong Pang[1*], Swee King Phang[1]

[1]School of Engineering, Taylor's University, Subang Jaya, 47500, Selangor, Malaysia.

*Corresponding author(s). E-mail(s): waileong.pang@taylors.edu.my;

Abstract

Fifth generation (5G) network slicing must admit heterogeneous requests across cooperating base stations under strict per-slice guarantees. This paper presents AXIOM, a multi-agent slice admission controller with a shared cluster encoder and one decision head per slice. We evaluate it under a pre-registered protocol in two simulators, one with static load and one with time-varying load and inter-cell coupling, against single-agent and independent deep Q-networks (DQNs), multi-agent proximal policy optimisation (MAPPO), and tuned threshold rules. AXIOM's advantage over learned baselines grows with cluster size. At 7, 19, and 37 gNodeBs (gNBs) it keeps Ultra-Reliable Low-Latency Communication (URLLC) and enhanced Mobile Broadband (eMBB) Service Level Agreement (SLA) compliance above 93%, while every learned baseline stays below 84% and only a known static rule does better. Its parameter count stays at 7,782 at any size. At three gNBs it reaches 98.9% of an oracle rule's reward under static load, and under coupling it beats the static rule by 39% and leads every learned baseline. Federated training shows no significant reward cost, though a moderate one remains possible. A three-parameter rule tuned with domain knowledge outperforms every learned policy under coupling at its tuning conditions, but AXIOM overtakes it when load rises beyond them. A single DQN over the joint state can match AXIOM in some settings, and graph attention adds nothing over a per-node encoder. Recomputation and pre-stated checks on fresh seeds confirm the main findings. A live OpenAirInterface (OAI) testbed validates only the control path, not AXIOM itself.

Keywords: Network Slicing, Slice Admission Control, Multi-Agent Reinforcement Learning, Graph Neural Networks, Federated Learning, Open RAN

M. P. Kunasegran: orcid.org/0009-0005-3169-1873. W. L. Pang: orcid.org/0000-0001-8407-5648. S. K. Phang: orcid.org/0000-0002-7877-8766.

1 Introduction

Network slicing virtualises the physical radio access network (RAN) to serve heterogeneous services under Ultra-Reliable Low-Latency Communication (URLLC), enhanced Mobile Broadband (eMBB), and massive Machine-Type Communication (mMTC) categories. Each slice carries distinct Service Level Agreement (SLA) targets that are in turn based on Quality of Service (QoS) thresholds. Our earlier work [16, 17] validated that a Deep Q-Network (DQN) admission controller can enforce SLAs for a single gNodeB (gNB) in a live OpenAirInterface (OAI) testbed. However, a real deployment spans multiple gNBs that share backhaul, spectrum planning, and a cluster-wide load-balance objective. Thus, slice admission at one gNB is rarely isolated, and coordinated admission is in fact a multi-gNB, multi-agent challenge. This research aims to address whether a shared, graph-structured view of the cluster could enable per-gNB agents to decide better than a single agent on a flattened state, as opposed to independent agents. Fig. 1 places the research premise in context.

This paper introduces AXIOM, an Adaptive Cross-cell Integrity-aware O-RAN Multi-agent slice-admission controller, where O-RAN denotes the Open Radio Access Network. AXIOM is a centralised-training, decentralised-execution multi-agent architecture that is built on a shared Graph Attention Network (GAT) [38] encoder over a multi-gNB topology, which feeds decentralised per-slice Q-heads. We deliver it together with the evaluation discipline needed to trust what it reports. Three gaps from our systematic review of 67 core studies motivate this pairing, as Section 2 details. First, no reviewed study checks whether its own outcome metric can score a degenerate policy as well as a correct one. Second, none replicates a headline architectural finding on an independent seed sample. Third, none tests whether a live testbed can reach

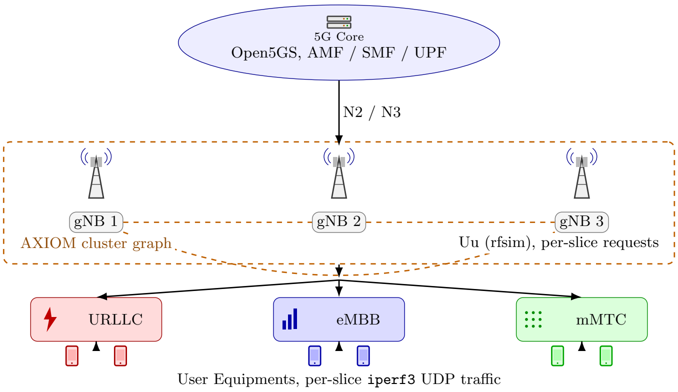

Fig. 1 Conceptual system view. A cluster of gNBs, coordinated by the shared AXIOM encoder and backhauled to a 5G core, serves user equipments (UEs) across three slices. AXIOM is evaluated offline; the live rig runs one gNB

the contention regime its offline claims assume. We address these gaps through a per-component realisation of the architecture, namely the GAT encoder, the Multi-Agent Reinforcement Learning (MARL) controller, the Federated Learning (FL) coordinator, and the Differential Privacy (DP) layer, each measured and isolated rather than folded into one end-to-end number. We apply a correctness-aware metric pair and a reproduction-versus-replication discipline throughout, and we add a live single-gNB deployment that tests, rather than assumes, whether the rig reaches genuine per-slice scarcity.

We evaluate AXIOM in two offline simulators of a gNB cluster. The first, Env A, has a static load in which a static rule that sheds every mMTC request is close to optimal. The second, Env B, adds time-varying load and inter-cell coupling, so that no fixed rule is optimal and neighbouring gNBs affect each other. We compare AXIOM with single-agent and independent deep Q-networks, with multi-agent proximal policy optimisation (MAPPO), and with tuned threshold rules. We also extend it to federated training with differential privacy, stress it under gNB dropout, demand spikes, and agent churn, and grow the cluster to 7, 19, and 37 gNBs under fully-connected, ring, and hex-grid adjacency. The contributions of this paper are as follows.

- AXIOM, a multi-agent controller with a shared cluster encoder and decentralised per-slice Q-heads, whose parameter count does not grow with the cluster. In Env A it keeps priority-slice compliance above 93% at 7, 19, and 37 gNBs, where both DQN baselines and MAPPO stay below 84%, and in Env B it leads every learned baseline at three gNBs.
- A harder environment with time-varying load and inter-cell coupling, in which learned policies beat static rules and a tuned threshold rule sets the bar they have yet to reach.
- An evaluation protocol for learned slice admission control that combines trivial-policy checks, per-slice correctness metrics, independent-seed replication, tuned rule baselines, and a pre-registered analysis. It caught seven evaluation failures in this work, and every main finding was then confirmed on fresh seeds.
- Clear bounds on the architecture's claims, namely that graph attention adds nothing over a per-node encoder and that per-step differential privacy gives only a weak guarantee.

2 Related Work

Our prior systematic review [18] screened 6,286 records down to 67 core studies from 2024 to 2026 on machine-learning-driven, user-centric distributed network slicing. The review organised them around three enabling pillars, namely topology-aware graph representation, distributed multi-agent control, and privacy-preserving federation. Each pillar is well developed on its own, but the three are rarely combined under realistic constraints, and never with a validated live deployment. No reviewed study checks whether its own evaluation metric can mask a degenerate policy, and none replicates a headline finding on an independent seed sample, which is the gap this paper fills.

Studies from the review's core sample show the state of each pillar. For admission control, Ojijo et al. [24] admit slice requests with a sequential twin actor-critic over a multi-dimensional state space and a distributed action space, and Tao et al. [34] use a digital twin to accelerate online deep reinforcement learning for admission. For multi-agent control, Doanis and Spyropoulos [8] make multi-agent DQNs sample-efficient for multi-domain inter-slice orchestration, Li et al. [20] propose a scalable multi-agent framework for dynamic RAN slicing, and ICE-CREAM [25] coordinates fully cooperative decentralised agents for energy-efficient RAN slicing. For graph representation, Hao et al. [11] combine a graph neural network with meta-learning to make bandwidth allocation scalable and transferable. For federation, Amiri et al. [4] split virtual network functions in O-RAN slicing with federated deep reinforcement learning, and Ming et al. [22] apply federated deep reinforcement learning to prediction-based slice mobility. Closest to our evaluation concern, Tsampazi et al. [37] design and evaluate many deep reinforcement learning agents for O-RAN in an automated pipeline, a systematic comparison that our trivial-policy and independent-seed checks extend.

AXIOM also sits inside the broader O-RAN effort to run learned control loops as applications over standard interfaces. Polese et al. [27] survey that architecture and its open problems, while ColO-RAN [26] and Tsampazi et al. [36] train and compare deep reinforcement learning controllers for slicing on large experimental platforms. Learned slice admission predates O-RAN, from market-driven admission of slice requests [5] to reinforcement learning for RAN slicing [30]. For cooperating agents, value factorisation such as QMIX [29] and centralised-critic policy gradients such as MAPPO [40] are the standard training schemes, and we compare against MAPPO directly. Constrained multi-agent learning has also been applied to Open RAN slicing at scale [41], and Khani et al. [13] survey deep reinforcement learning for slice admission in cloud RAN. Within this journal, recent work applies actor-critic learning to joint resource allocation in RAN slicing [15], double deep Q-networks to next-generation networks [12], and delay-aware slice allocation to cloud RAN [3]. Other work in the journal applies federated learning to 6G slice resource allocation [14] and multi-agent reinforcement learning to device-to-device relay selection [32]. None of these evaluates against a trivial-policy check or replicates its headline result on an independent seed sample. AXIOM instead shares one graph encoder across agents and trains every pending request as its own temporal-difference (TD) sample, for the reason Section 4.1 gives.

Several recent works come close on a single axis of this problem. Shao et al. [31] motivate our GAT encoder choice by showing that graph attention strengthens inter-agent cooperation over plain DQN and Advantage Actor-Critic (A2C) baselines. Ahmadi et al. [2] study topology generalisation through graph attention, which complements but does not answer the fixed-topology evaluation-integrity question we ask in Section 5.2. Sulaiman et al. [33] are the closest prior art to our joint slicing-and-admission-control problem. None of these three checks whether its reward-optimal policy could be mistaken for a degenerate one, and none replicates a headline result on an independent seed sample. Table 1 sets AXIOM beside the six closest works. Two of them study admission control, four use a graph encoder, and none describes federated training, so none combines the three as AXIOM does.

Table 1 Closest prior work, as described in each paper's title and abstract

Work	Problem	Multi-agent	Graph encoder	Federated
Shao et al. [31]	Slicing resource management in dense cells	Yes	Attention	Not described
Sulaiman et al. [33]	Joint slicing and slice admission in cloud RAN	Yes	Not described	Not described
Ahmadi et al. [2]	RAN and edge-computing slicing with admission control	Yes	Attention	Not described
Zangooei et al. [41]	SLA-constrained RAN slicing in Open RAN	Yes	Not described	Not described
Mohajer et al. [23]	SLA-aware task offloading and service orchestration	Yes	Attention	Not described
Bouroudi et al. [6]	Energy-efficient network-function placement	Yes	Graph neural network	Not described
AXIOM	Per-slice admission across a gNB cluster	Yes	Attention or per-node MLP	Federated averaging, optional DP

Robustness studies share our concern but stop short of it. Tashman and Cherkaoui [35] and Fatehi et al. [9] study robustness to a disruptive condition, as our disruption campaign in Section 5.6 does, yet neither checks whether the reported robustness metric is trustworthy and neither replicates a finding on an independent sample. That is the gap our independent-seed replication closes. Zhang et al. [42] are the closest periodic-synchronisation precedent, and Yasin et al. [39] are the closest precedent for federated learning with differential privacy. Their threat model differs from ours, since they secure the controller rather than characterise a control policy's own privacy-utility trade-off, which our σ sweep in Section 5.5 reports, and neither adds a correctness-aware check on whether federating or privatising changed decision quality. A further, closely related set [6, 10, 23, 28] combines two of the three pillars, yet like every work above, each evaluates on its own reported outcome metric, computed once, on one seed or run.

Rather than adding a new point in the GAT, MARL, and FL design space, this paper fills that gap with a demonstration on AXIOM, extended to federation, privacy, and disruption. It shows that two checks are both necessary, namely testing a metric's own trustworthiness and replicating a finding on an independent sample. Skipping either changes what a reader may believe the resulting numbers mean.

3 System Model and Problem Formulation

Fig. 2 provides an overview of the system. The AXIOM controller is evaluated offline in two shared-budget simulation environments, Env A and Env B. Evaluation safeguards and a pre-registered analysis then check every reported result. The deployment pipeline is then checked on a live single-gNB testbed, which Section S9 of Online Resource 1 describes.

We consider a cluster of N=3 gNBs, each serving the same three slices, URLLC, eMBB, and mMTC, under a shared per-gNB Physical Resource Block (PRB) budget B. An admission decision is a PRB-ceiling adjustment through the `slicing_control_m` minimum and maximum ratio, rather than a literal per-flow accept or reject, since the real OAI xApp control interface exposes only ratio ceilings. No real multi-gNB physical topology is available to us, so we take the least assumption-laden choice and treat the cluster as fully connected, as a modelling choice rather than a measured fact. At each control step, every gNB holds zero or more pending admission requests, each tagged with a slice identity, and an agent decides whether to accept or reject for each request independently, conditioned on the current cluster state. The discrete action space matches our single-gNB work exactly at `action_dim`=2, so any coordination benefit found here is due to the architecture and not a richer action space.

The step reward follows our prior single-gNB formulation. It has a per-slice service term net of SLA-violation penalties, a congestion charge on admitted volume, and a load-imbalance penalty across gNBs. The step reward r_t sums over slices indexed by k. It uses priority weight ω_k, per-accept reward R_k, admitted count n_k, violation penalty λ_k, and the indicator $\mathbf{1}[\cdot]$ equal to one exactly when slice k is in SLA violation that step,

$$r_t = \sum_k \big(\omega_k R_k\, n_k - \lambda_k\, \mathbf{1}[v_k]\big) \;-\; \kappa\, C_t \|\mathbf{a}_t\|_1 \;-\; \beta\,(1 - F_t), \tag{1}$$

where $\|\mathbf{a}_t\|_1$ is the total admitted this step, C_t the cluster-mean congestion level, κ the congestion coefficient, $F_t \in [0,1]$ the min-over-max PRB-utilisation fairness ratio, and β the load-balance weight. The violation flag v_k follows from a continuous SLA margin m_k. The correctness-aware metrics of Section 4.4 also build on this margin. We take it as the worse of the queue and loss channels,

$$m_k = \min\Big(1 - \tilde{q}_k,\ 1 - \tfrac{\ell_k}{\ell_k^{\max}}\Big), \qquad v_k \Leftrightarrow m_k \le 0, \tag{2}$$

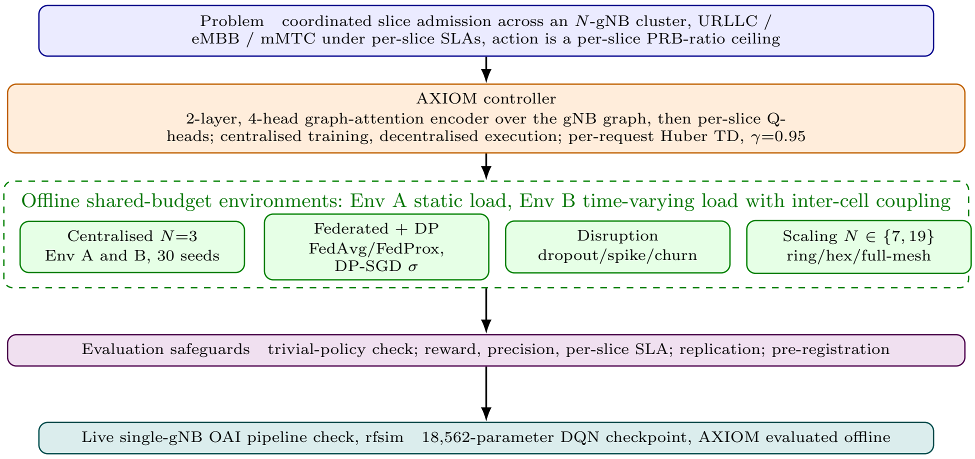
Problem coordinated slice admission across an N-gNB cluster, URLLC / eMBB / mMTC under per-slice SLAs, action is a per-slice PRB-ratio ceiling
AXIOM controller
2-layer, 4-head graph-attention encoder over the gNB graph, then per-slice Q-heads; centralised training, decentralised execution; per-request Huber TD, γ=0.95
Offline shared-budget environments: Env A static load, Env B time-varying load with inter-cell coupling
Centralised N=3
Env A and B, 30 seeds
Federated + DP
FedAvg/FedProx,
DP-SGD σ
Disruption
dropout/spike/churn
Scaling $N \in \{7, 19\}$
ring/hex/full-mesh
Evaluation safeguards trivial-policy check; reward, precision, per-slice SLA; replication; pre-registration
Live single-gNB OAI pipeline check, rfsim 18,562-parameter DQN checkpoint, AXIOM evaluated offline

Fig. 2 Overview of the research. An AXIOM controller is evaluated across four offline campaigns in two shared-budget environments, every result checked by evaluation safeguards, with the deployment pipeline checked on a live single-gNB testbed

with $\tilde{q}_k$ the slice backlog relative to its budget, and ℓ_k and $\ell_k^{\max}$ its loss proxy and loss budget. The weights are set so that, under congestion, the intended policy is differentiated shedding, which keeps accepting URLLC and eMBB and sheds mMTC.

Every result uses an offline, closed-loop simulation of the cluster, except the bounded live anchor of Section 5.9. Each gNB has a budget of $B{=}100$ PRBs shared by its three slices. Each slice's mean offered demand is 1.05 times its nominal PRB share of 30, 60, and 10 PRBs for URLLC, eMBB, and mMTC. Aggregate demand therefore exceeds the budget by about five per cent, while URLLC and eMBB demand stays below their own caps of 33 and 66 PRBs. Whenever the slices' admitted demand exceeds B, they share B in proportion to that demand, so every admitted mMTC request takes resources from URLLC and eMBB. Unserved demand carries forward as backlog within an episode, which drives the SLA margin of Eq. (2). Every episode starts with empty queues, and per-gNB load multipliers drawn from $[0.9, 1.0]$ keep the cluster heterogeneous. We also use a second, harder environment, Env B, which keeps Env A and adds two mechanisms. First, each gNB's mean demand follows $0.95\,(1 + 0.3\sin(2\pi(s + \psi_i)/60))$ times its nominal shares, where s is the step within the episode and the phase ψ_i is redrawn for every gNB at every episode, so demand swings between about 0.67 and 1.24 times the budget. Second, a gNB's usable budget shrinks with its neighbours' load on the previous step, $B_i(t) = B\,(1{-}0.15\,\bar{u}_{\mathcal{N}(i)}(t{-}1))$, where $\bar{u}_{\mathcal{N}(i)}$ is the mean PRB utilisation of i's neighbours in the same adjacency graph the encoder reads. This is a first-order model of inter-cell interference, which a gNB observes only through its own queues. The reward weights are unchanged from Env A. In Env B shedding every mMTC request is no longer the best rule, since it leaves URLLC compliance at 62.5% at three gNBs, whereas a rule that also sheds eMBB when the URLLC queue builds keeps it at 98.9%. Both constants were calibrated on pilot seeds with rules only, before any reported seed was run. Per-slice compliance counts a slice as compliant only when every gNB meets it, so in Env B it saturates at nineteen gNBs. Env B is therefore run at three and seven gNBs, and the nineteen-gNB evidence comes from Env A.

4 The AXIOM Architecture

AXIOM differs from its two DQN baselines in exactly what each policy sees and shares. AXIOM runs one graph-attention encoder over the whole gNB cluster, trains that encoder jointly from every agent's experience, and reads out one dedicated Q-head per slice, so every gNB decides from a shared view of the cluster that the adjacency graph shapes. The single-agent DQN keeps one controller over a flattened joint state, with no per-gNB structure and no graph. The independent DQN gives each gNB its own learner that sees only local features and never shares parameters or reads the cluster topology. The two baselines are therefore ablations of AXIOM that remove the shared encoder and the graph input together, so any gap between them and AXIOM measures what those components add jointly. A third variant that keeps everything except attention isolates the graph's own contribution in Section 5.3.

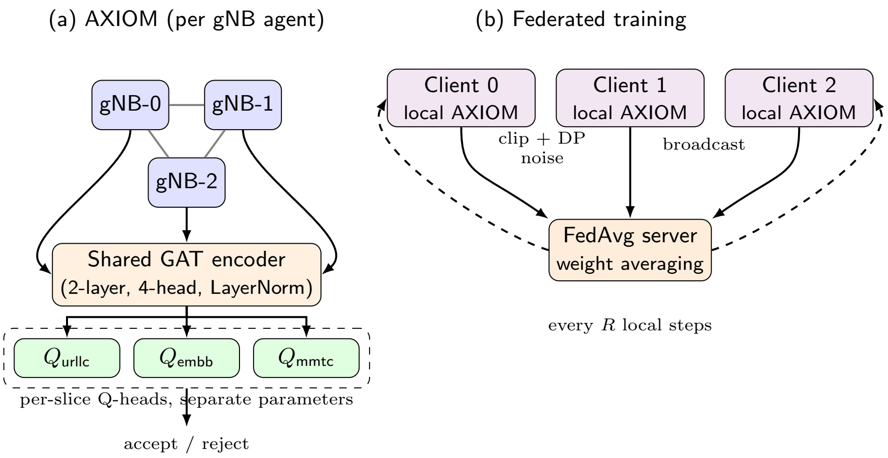
(a) AXIOM (per gNB agent)
gNB-0
gNB-1
gNB-2
Shared GAT encoder
(2-layer, 4-head, LayerNorm)
Q_{urllc}
Q_{embb}
Q_{mmtc}
per-slice Q-heads, separate parameters
accept / reject
(b) Federated training
Client 0
local AXIOM
Client 1
local AXIOM
Client 2
local AXIOM
clip + DP noise
broadcast
FedAvg server
weight averaging
every R local steps

Fig. 3 AXIOM architecture. a, A shared graph-attention encoder over the fully-connected 3-gNB topology produces one embedding per node, consumed at execution time by per-slice Q-heads using only that node's row. b, The federated variant, each gNB a client synchronised by periodic FedAvg with an optional DP-SGD clip-and-noise step

4.1 Shared Encoder, Decentralised Q-Heads

Fig. 3a shows the architecture. At every control step, each gNB forms its own per-slice feature row $\mathbf{x}_i$. The three features are PRB-used-ratio, congestion-level, and queue-length-norm. These rows stack into a joint node-feature matrix $\mathbf{X} \in \mathbb{R}^{N\times d}$ over the N gNB nodes, with d features each. A two-layer, H=4-head GAT [38] produces one embedding per gNB. Each head h decomposes additive attention rather than scoring a concatenation. It uses a learned source vector $\mathbf{a}_s^h$ and destination vector $\mathbf{a}_d^h$, applied to the shared linear projection $\mathbf{W}^h$. Here LeakyReLU is the leaky rectifier of slope 0.2, and $\mathcal{N}_i$ is the set of neighbours of node i under the adjacency mask,

$$\varepsilon_{ij}^h = \mathrm{LeakyReLU}\big(\mathbf{a}_s^{h\top}\mathbf{W}^h\mathbf{x}_i + \mathbf{a}_d^{h\top}\mathbf{W}^h\mathbf{x}_j\big), \qquad \alpha_{ij}^h = \frac{\exp \varepsilon_{ij}^h}{\sum_{l\in\mathcal{N}_i}\exp \varepsilon_{il}^h}. \tag{3}$$

Here ε_{ij}^h is the unnormalised attention score from node i to neighbour j in head h, and α_{ij}^h is its normalisation over i's neighbours, with l ranging over $\mathcal{N}_i$. Hidden layers concatenate the H heads under an exponential linear unit (ELU) nonlinearity, while the final layer averages them. A per-layer normalisation operator LN$(\cdot)$ then rescales each node's embedding to zero mean and unit variance, and produces the final per-gNB embedding $\mathbf{z}_i$,

$$\mathbf{z}_i = \mathrm{LN}\Big(\tfrac{1}{H}\textstyle\sum_{h=1}^{H}\sum_{j\in\mathcal{N}_i}\alpha_{ij}^h\,\mathbf{W}^h\mathbf{x}_j\Big). \tag{4}$$

Centralisation comes from this encoder, whose parameters every agent's loss updates jointly, so it sees the full cluster state during training. Decentralised execution comes next, since at action-selection time each agent consumes only its own embedding row $\mathbf{z}_i$ to decide its own pending requests.

Algorithm 1 AXIOM: centralised training, decentralised execution

```
Require: encoder $f_\theta$, per-slice Q-heads $\{Q_{\phi_k}\}$, targets $f_{\theta^-}, \{Q_{\phi_k^-}\}$, buffer $\mathcal{D}$, adjacency $\mathbf{A}$, discount $\gamma$, sync period $T_\mathrm{s}$
for each step $t$, observing joint node features $\mathbf{X}_t$ do
    $\mathbf{E}_t \leftarrow f_\theta(\mathbf{X}_t, \mathbf{A})$ {centralised, one row per gNB}
    for each pending request at gNB $i$, slice $k$, pick $a \leftarrow \arg\max_a Q_{\phi_k}([\mathbf{E}_t[i], \mathbf{X}_t[i]])[a]$, $\epsilon$-greedy {decentralised, own row only}
    step env, store $(\mathbf{X}_t, \mathbf{X}_{t+1}, \{(i, k, a)\}, r_t, \text{done})$ in $\mathcal{D}$
    if $|\mathcal{D}| \geq$ warmup then
        sample minibatch $\mathcal{M}$; $\mathbf{E}_\mathcal{M} \leftarrow f_\theta(\mathbf{X}_\mathcal{M}, \mathbf{A})$, $\mathbf{E}'_\mathcal{M} \leftarrow f_{\theta^-}(\mathbf{X}'_\mathcal{M}, \mathbf{A})$
        per request, $y \leftarrow r + \gamma(1-\text{done}) \max_{a'} Q_{\phi_k^-}(\mathbf{E}'_\mathcal{M}[i])[a']$
        $\mathcal{L} \leftarrow$ mean per-request Huber loss, never summed per step; update $\theta, \{\phi_k\}$, clip grad-norm 1.0
    end if
    every $T_\mathrm{s}$ episodes, $\theta^- \leftarrow \theta$, $\phi_k^- \leftarrow \phi_k$
end for
```

The `LayerNorm` and the per-slice heads are deliberate design choices. With a single shared Q-head, all three slices update the same parameters, and URLLC's large priority weight produces gradients that swamp mMTC's much smaller signal, a standard shared-parameter class imbalance. One small multi-layer perceptron (MLP) per slice, selected by the context one-hot, gives each slice its own parameters, which Section 5.3 confirms matters at nineteen gNBs, and the `LayerNorm` stops an embedding from encoding congestion through its magnitude alone. That normalisation also hides the absolute queue and congestion levels on which a state-dependent admission rule depends, so each node's raw features are concatenated onto its normalised embedding before the Q-heads, $Q_{\phi_k}([\mathbf{z}_i, \mathbf{x}_i])$, and below we write $\mathbf{z}_i$ for that concatenation. This skip connection, together with a learning rate of 10^{-3} in place of the earlier 10^{-4}, is the only change from the original AXIOM design. Both were chosen on pilot seeds of the time-varying environment of Section 3, where the earlier AXIOM learned only the static shedding rule, and both were fixed before any reported seed was run. A seed here is one independent random initialisation of network weights and environment stream, and we report every seed rather than a single best run.

Algorithm 1 states the training loop, where the encoder updates from every request in a sampled minibatch regardless of gNB or slice. This is centralised training, while action selection at rollout time consults only the acting agent's own embedding row, which is decentralised execution. Every pending request at gNB i, slice k is its own temporal-difference sample, and we never sum requests across a step before the loss. For a request with immediate reward r, next-state embedding $\mathbf{z}'_i$, and candidate next action a', the bootstrapped target y at discount γ and the minibatch loss are

$$y = r + \gamma\,(1 - \text{done}) \max_{a'} Q_{\phi_k^-}\big(\mathbf{z}'_i\big)[a'], \qquad \mathcal{L} = \frac{1}{|\mathcal{M}|} \sum_{(i,k,a)\in\mathcal{M}} \ell_\delta\Big(Q_{\phi_k}(\mathbf{z}_i)[a] - y\Big), \quad (5)$$

where Q_{ϕ_k} is slice k's online Q-head, $Q_{\phi_k^-}$ its target copy, done the episode-done flag, and $\mathcal{M}$ the sampled minibatch of requests. We use the Huber loss ℓ_δ in place of squared error, which keeps the deeper encoder stable under long bootstraps. An earlier variant passed each gNB's summed per-request values through a QMIX-style monotonic mixer [29]. Because the number of pending requests changes from step to step, that mixed target changed scale with the request count, and the mean training loss diverged, rising from about 76 over the first fifth of training steps to about 409 over the last fifth. Treating each request as its own sample removes that dependence, although Section 5.3 shows that a plain summed target works as well, so the per-request scheme is a simplification rather than a source of AXIOM's advantage.

4.2 Federated Training with Differential Privacy

Fig. 3b shows the federated variant. In this variant, each gNB becomes a federated-learning client. It holds a private copy of the same network and trains only on its own requests. Transitions are partitioned by owning agent, so no client sees another's raw admission data. Each client's GAT encoder still consumes the full joint node-feature snapshot. Every N_{loc} local steps, the server averages all clients' online weights by federated averaging (FedAvg) [21] and broadcasts the result back. We also implement FedProx [19], which adds a proximal term that pulls local weights toward the last broadcast global model. We sweep it at $\mu \in \{0.01, 0.1, 1.0\}$ on three seeds. FedProx gives no consistent improvement over FedAvg, as Section 5.5 reports, so we keep FedAvg.

Differential privacy (DP) bounds how much any one client's training data can change what the shared model reveals. We apply it as a differentially private stochastic gradient descent (DP-SGD) step [1]. Each client clips its minibatch gradient to a fixed norm C_{dp} and, when privacy is enabled, adds Gaussian noise of standard deviation σC_{dp} before its optimiser step. The clip applies to the whole minibatch gradient rather than to each example, so changing any of the client's data in one step moves the clipped gradient by at most $2C_{\mathrm{dp}}$, and the guarantee covers the client's whole step batch. We account for it with zero-concentrated differential privacy (zCDP) [7], where ε and δ are the standard privacy-loss and failure-probability parameters. Composing the Gaussian mechanism over the m noisy steps a client takes gives

$$\rho = \frac{2m}{\sigma^2}, \qquad \varepsilon(\delta) = \rho + 2\sqrt{\rho \ln(1/\delta)}. \tag{6}$$

We credit no amplification by subsampling, because minibatches come from a growing replay buffer that reuses records, so Eq. (6) is an upper bound. The clip is applied whether or not noise is added, so a $\sigma{=}0$ run and a $\sigma{>}0$ run differ only in the injected noise. The noise protects each client's own admission decisions and rewards, but not its congestion features, which every client's encoder reads from the shared node-feature snapshot by design. Each federated round therefore runs N_{loc} local Huber-TD steps per client with an optional FedProx proximal term $\frac{\mu}{2}\|\theta_i - \bar{\theta}\|^2$, clips and optionally perturbs the gradient, and closes with a FedAvg average that is broadcast back.

4.3 Experimental Setup

Every campaign uses 300 training and 50 evaluation episodes per seed, and each seed is evaluated on a held-out environment stream. The three-gNB campaigns use 30 seeds, 900 to 929, in each environment, with a disjoint replication sample of seeds 1000 to 1009. The scaling campaigns use 15 seeds per cell at seven gNBs and 30 per cell at nineteen, and the thirty-seven-gNB extension uses 30 seeds on the hex layout and 15 on each other layout. We compare AXIOM with three learned baselines and several rules. The single-agent DQN is one controller over the flattened joint state. The independent DQN gives each gNB its own DQN with no parameter sharing and no topology information. Both DQN policies are ablations, deliberately reduced variants with a component removed so that its effect can be isolated. MAPPO [40] is a published multi-agent baseline with a shared actor over each gNB's own features and a centralised critic over the joint state. It trains on-policy with proximal policy optimisation, for the same 300 episodes and on the same global reward. Its learning rate, epochs, and minibatch size were tuned on pilot seeds only, because untuned it collapsed into accepting every request in four of five pilot seeds, and a wider grid in Section 5.3 confirms that AXIOM's lead does not depend on this choice.

Rules need no learning, and accept-all simply admits every request. The oracle rule of Env A sheds mMTC whenever its gNB's congestion level reaches a threshold τ, chosen on training streams only, and the selected value is τ=0, so the rule sheds every mMTC request. We call it the oracle rule because it is the best rule of its family on training streams. It is a strong reference rather than a proven optimum, since across our Env A samples learned policies exceeded it in 28 of 450 seed-level evaluations, by at most 0.40 reward per step. In Env B we add two tuned rule families, each chosen on training streams over a grid of thresholds and evaluated on the held-out streams. The local family sheds mMTC on the gNB's own congestion or priority-slice queue and sheds eMBB on its URLLC queue, while the neighbour-aware family also reads the mean congestion of the gNB's neighbours. Two further arms isolate AXIOM's components, namely the DQN baselines at the lower learning rate of 10^{-4}, and an AXIOM variant whose graph encoder is replaced by a per-node MLP of the same width that ignores the adjacency.

All learned arms share the discount factor $\gamma = 0.95$, the training budget, and the exploration schedule, and AXIOM and the DQN baselines train at learning rate 10^{-3}. Table 2 lists the simulation and training parameters, and Section S10 of Online Resource 1 gives every remaining hyperparameter and the design of the two sensitivity studies, which change the load and the slice weights in Env A and the learning rate and encoder width in Env B.

All confidence intervals (CIs) are 95% bootstrap percentile intervals over 10,000 resamples. Paired comparisons use the Wilcoxon signed-rank test, and each family of comparisons against AXIOM is Holm-corrected for multiple testing. An AI assistant, Anthropic's Claude, was used to help write the analysis and plotting scripts and to edit the manuscript text. The authors made all research decisions and take full responsibility for the content of this paper.

The settings, seeds, and hypotheses of the final campaign were fixed in a dated pre-registration, committed before any reported seed was run. Pre-registration here means

Table 2 Simulation and training parameters, shared by both environments unless stated

Parameter	Value
PRB budget per gNB, B	100
Nominal share, floor, cap, in PRB ratio	URLLC 30, 5, 33; eMBB 60, 10, 66; mMTC 10, 5, 11
Priority weight ω_k	URLLC 12.0, eMBB 3.5, mMTC 0.3
Per-accept reward R_k	1.0 for every slice
Violation penalty λ_k	URLLC 10.0, eMBB 5.0, mMTC 2.5
Latency and loss budgets	URLLC 15 ms, 0.8%; eMBB 45 ms, 2.0%; mMTC 65 ms, 3.5%
Congestion coefficient κ, load-balance weight β	1.5, 1.0
Episode	60 steps of 5 s; queues emptied at each episode start
Requests per step	N, each with a uniformly random slice and gNB; at most $4N$ pending
Offered demand per slice	1.05× nominal share, per-step noise $0.005B$, per-gNB multiplier in $[0.9, 1.0]$
Env B load and coupling	$0.95\,(1 + 0.3\sin(2\pi(s + \psi_i)/60))$, $B_i(t) = B\,(1 - 0.15\,\bar{u}_{\mathcal{N}(i)}(t-1))$
Admission gate step	5 PRB-ratio points per accept or reject
Training and evaluation	300 and 50 episodes per seed, held-out evaluation stream
Value-based learners	Adam, learning rate 10^{-3}, $\gamma = 0.95$, ϵ from 1.0 to 0.05 at ×0.985 per episode, replay 10,000, minibatch 16, warm-up 32, target sync every 10 episodes
AXIOM network	Two-layer, four-head GAT, hidden and output 16, raw-feature skip, per-slice Q-heads of hidden 32, Huber loss, gradient clip 1.0
DQN baselines	Two hidden layers of 128, squared-error loss
MAPPO	Actor and critic learning rate 10^{-3}, 10 epochs, minibatch 64, clip 0.2, entropy 0.01, GAE $\lambda = 0.95$

that each analysis plan was committed to the project's version-control history before the runs it governs, rather than lodged with an external registry, and Section S13 of Online Resource 1 lists the commit identifiers. It includes the change to AXIOM's learning rate and skip connection and the constants of Env B, both chosen on pilot seeds 2000 to 2009. It states four hypotheses, all fixed before any result. H1 is that AXIOM beats both trivial rules in Env B. H2 covers how AXIOM compares with every learned baseline and tuned rule in Env B. H3 asks whether graph attention beats the per-node encoder in Env B at seven gNBs. H4 covers how AXIOM compares with every baseline at nineteen gNBs in Env A, over 30 seeds per layout.

A pre-registered validation screen checks every trained checkpoint of every learned arm on its own training stream before evaluation, and every seed is reported whether or not it passes, with the screen's criteria in Section S10 of Online Resource 1.

4.4 Evaluation Safeguards

Three safeguards make the results below trustworthy, and Table 3 lists the failure each one caught during this work. A standard pipeline would have reported every one of them as a clean result.

The first is a trivial-policy check, in which we evaluate the simplest rules the environment admits, accepting every request and shedding one slice outright, before trusting any learned policy. It exposed an earlier simulator in which accepting every

Table 3 Evaluation failures this paper's safeguards caught during the work, none of which a standard pipeline would have surfaced

Failure	Missed by	Caught by	Outcome
Inert simulator (§3)	Learned-policy metrics	Accept-all baseline	Shared-budget rerun
No-op scored as fine	All-slice compliance	Precision, per-slice SLA	Collapse made visible
Repro. gap (Online Resource 1)	Single-sample claim	Repro-vs-replication	Churn claim retracted
Backlog carry-over (§4.4)	Per-seed means	Replication, step traces	Episode reset, rerun
Static AXIOM in Env B (§4.1)	Env A results	Env B pilot seeds	Faster learning rate, skip
Screen miscalibrated (§4.3)	Env A screen design	Pilot screen check	Pre-registered amendment
Scheduler floor (§5.9)	Cross-layer floor	Direct instrumentation	100k+ readings clear

request was optimal, and the shared-budget simulator of Section 3 removes that loophole.

The second is a correctness-aware set of metrics, computed from data every run already logs. Mean reward per step is the training objective. Block precision is the fraction of blocked requests that correctly target mMTC, the slice the reward calibration makes reject-optimal. Writing b_{mMTC} and b for the mMTC-targeted and total blocks over an evaluation, it is $\pi = b_{\mathrm{mMTC}}/b$ for $b > 0$ and undefined for $b = 0$, so a seed that never blocks is counted as a collapse rather than scored as a success. Per-slice SLA compliance then shows whether the shedding actually protects URLLC and eMBB. We do not use the common all-slice compliance metric, because correct shedding gives up mMTC's own SLA by design, so that metric scores a correct policy and a failed one alike. Block precision applies to Env A, where mMTC is the only slice the reward makes worth shedding, and Env B is judged by reward and per-slice compliance, since its best rules also shed eMBB.

The third is a reproduction-versus-replication discipline, in which we recompute every headline comparison on an independent, disjoint seed sample and report both. It caught queue backlog that carried over between episodes, and every result in this paper comes from the corrected simulator, which empties all queues at the start of each episode.

5 Results

We compare AXIOM with the three learned baselines and the rules of Section 4.3 in both environments.

The comparison rests on four quantities, the first being mean reward per step, the objective the policies train on, so higher is better. Block precision is the share of a policy's rejections that land on mMTC. Per-slice SLA compliance shows whether a policy's shedding actually protects URLLC and eMBB, and we summarise it as priority-slice compliance, the mean of the two. Collapse names a failure mode in which a trained policy stops discriminating and simply accepts every request. Table 4 sets out how the four learned policies stand, and the subsections that follow give the evidence.

AXIOM's clearest advantage appears as the cluster grows. In Env A it keeps priority-slice compliance above 93% from seven to thirty-seven gNBs while every baseline falls to between 32% and 76%, with a parameter count that does not grow with the cluster. It also leads every learned baseline in Env B at three gNBs. Its advantage has limits, since a tuned threshold rule beats every learned policy in Env B and the single-agent DQN keeps pace with AXIOM in Env A at three gNBs and in Env B at seven.

Table 4 How the four learned policies compare. Env A has static load, Env B time-varying load with inter-cell coupling. Priority SLA is the mean of URLLC and eMBB compliance

Property	AXIOM	Single-agent DQN	Independent DQN	MAPPO
Design	Shared encoder, per-slice heads	One flattened controller	One learner per gNB	Shared actor, central critic
Parameters at execution	7,782 at any size	21,250 to 64,770	55,302 to 682,058	5,122
Env A reward, 3 gNBs	12.70, 98.9% of oracle	12.39	11.59	10.01
Env A priority SLA, 7, 19, 37 gNBs	97.1, 97.3, 93.7%	76.2, 65.3, 58.7%	74.6, 56.4, 32.1%	67.7, 60.6, 56.7%
Collapse, Env A, 19 and 37 gNBs	1/90 and 6/60	84/90 and 26/30	3/90 and 3/30	0/90 and 0/30
Env B reward, 3 gNBs	8.83	7.49	7.79	6.97
Env B priority SLA, 7 gNBs	35.1%	44.3%	39.9%	35.9%
Federated, noise-free	Cost +0.49 [−0.08, 1.09]	Not applicable	Not applicable	Not applicable

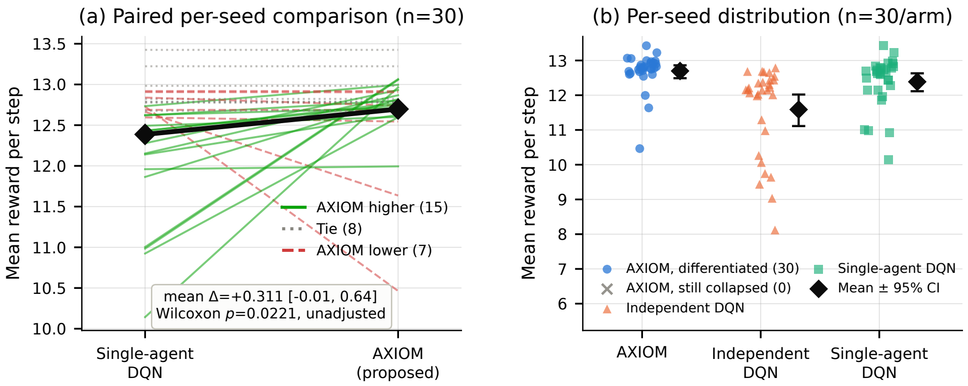

Fig. 4 Env A, three gNBs, per-seed evidence. a, Paired per-seed mean reward per step of AXIOM versus single-agent DQN, coloured by win, tie, or loss. b, Per-seed reward distribution for AXIOM and the two DQN baselines

Table 5 Env A, three gNBs, 30 seeds per arm, 95% bootstrap CIs. Precision averages the seeds that block at all. SLA columns give per-slice compliance, and LR is the learning rate

Arm	Reward/step	Precision	Collapsed	URLLC	eMBB
Oracle rule	12.835 [12.77, 12.91]	1.000	0/30	1.000	1.000
AXIOM	12.698 [12.49, 12.86]	0.992	0/30	0.998	0.979
AXIOM, MLP encoder	12.640 [12.46, 12.79]	0.986	0/30	1.000	0.976
Single-agent DQN	12.387 [12.11, 12.63]	0.990	0/30	0.994	0.941
at LR 10^{-4}	11.477 [10.61, 12.09]	0.944	0/30	0.979	0.866
Independent DQN	11.587 [11.11, 12.02]	0.962	0/30	0.993	0.865
at LR 10^{-4}	11.119 [10.73, 11.50]	0.878	0/30	0.985	0.794
MAPPO	10.013 [9.30, 10.73]	0.707	2/30	0.978	0.576

5.1 Static Load, Three gNBs

Table 5 and Fig. 4 report the thirty-seed campaign in Env A. AXIOM never collapses, 99.2% of its blocks land on mMTC, and it keeps URLLC and eMBB at 99.8% and 97.9% SLA compliance. Its mean reward of 12.698 per step is 98.9% of the oracle rule's 12.835, and the two tie exactly in 22 of the 30 seeds. AXIOM leads the independent DQN by +1.110 per step, with 95% CI [0.60, 1.65], and MAPPO by +2.685, with 95% CI [2.01, 3.35], both at Holm-adjusted $p \leq 0.0002$. Its lead over the single-agent DQN, +0.311 with 95% CI [−0.01, 0.64], does not survive correction on thirty seeds, at $p = 0.066$. MAPPO trails both DQN baselines here, because it sheds eMBB as well as mMTC, which lets eMBB compliance fall to 57.6%.

A disjoint replication on seeds 1000 to 1009 is stronger for AXIOM. There it leads the single-agent DQN, the independent DQN, and MAPPO by +1.083, +0.799, and +3.806 per step at Holm-adjusted $p \leq 0.023$, while sitting only 0.018 below the oracle. Pooled over all 40 seeds, AXIOM leads the three baselines by +0.504, +1.032, and +2.965 at Holm-adjusted $p \leq 0.002$, and falls 0.108 short of the oracle, a small gap that is significant at $p = 0.034$. Fresh seeds in Section 5.8 do not reproduce the lead over the single-agent DQN, however.

5.2 Cluster-Size Scaling

We grow the cluster in Env A from three gNBs to seven and nineteen under three layouts, fully-connected, ring, and hex-grid. Every gNB keeps its own shared 100-PRB budget and its own demand, so per-gNB load stays the same as the cluster grows. Each cell runs 15 seeds at seven gNBs and 30 at nineteen. A pre-registered extension then grows the cluster to thirty-seven gNBs, the three-ring hex cluster. The independent DQN ignores the layout, so it gives identical results under all three layouts, which checks that the layout changes nothing else. The single-agent DQN also ignores the layout, yet its results vary between layouts in some seeds, because its training is not bit-reproducible, for the reason Section 5.8 gives. We test the per-layout gaps on reward and on priority-slice compliance, Holm-correcting the 18 comparisons at each size.

At seven gNBs AXIOM keeps URLLC and eMBB at 99.8% and 94.4%, against 91.4% and 61.0% for the single-agent DQN, 94.2% and 55.0% for the independent DQN, and 87.0% and 48.4% for MAPPO. On fresh seeds the two DQN baselines reach

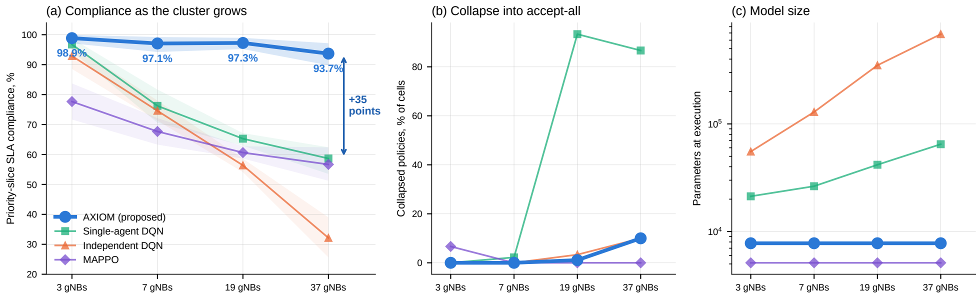

Fig. 5 Env A as the cluster grows, all layouts pooled. a, Priority-slice compliance, mean and 95% CI. b, Share of trained policies that collapse into accept-all. c, Parameters used at execution

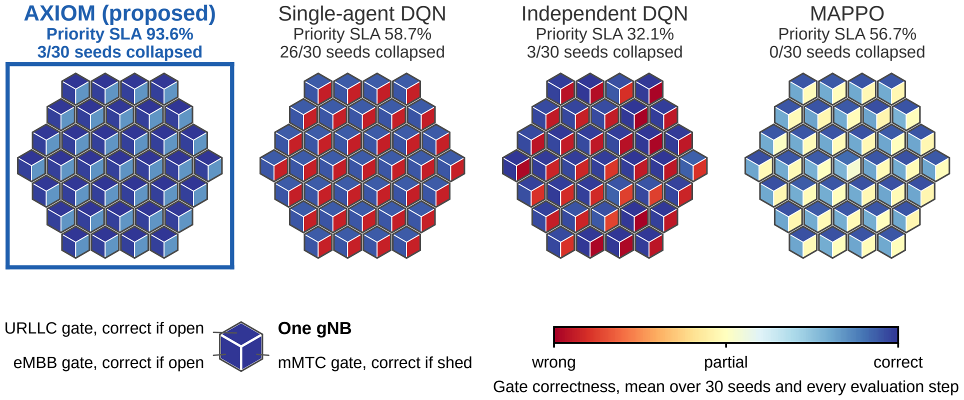
AXIOM (proposed)
Priority SLA 93.6%
3/30 seeds collapsed
Single-agent DQN
Priority SLA 58.7%
26/30 seeds collapsed
Independent DQN
Priority SLA 32.1%
3/30 seeds collapsed
MAPPO
Priority SLA 56.7%
0/30 seeds collapsed
URLLC gate, correct if open
eMBB gate, correct if open
One gNB
mMTC gate, correct if shed
wrong
partial
correct
Gate correctness, mean over 30 seeds and every evaluation step

Fig. 6 Decision map of the 37-gNB hex cluster in Env A over 30 seeds. Each hexagon is a gNB at its true position, split into its URLLC, eMBB, and mMTC admission gates and coloured by how correct each gate is

up to 83.3%, as Section 5.8 reports. Under every layout it leads all three baselines on priority-slice compliance by 17.9 to 29.9 percentage points and on cluster reward by 3.40 to 7.07 per step. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.011$.

At nineteen gNBs the gaps widen, and they answer hypothesis H4. AXIOM keeps URLLC and eMBB at 99.4% and 95.1%, a pooled priority-slice compliance of 97.3% with 95% CI [95.2%, 99.0%]. The single-agent DQN, the independent DQN, and MAPPO keep 65.3%, 56.4%, and 60.6%. Under every layout AXIOM leads every baseline by 31.0 to 42.1 percentage points of compliance and by 10.7 to 18.6 reward per step, winning 28 to 30 of 30 seeds. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.0001$. Fig. 5b shows the single-agent DQN collapsing into accepting every request in 84 of its 90 cells, against 3 for the independent DQN, 1 for AXIOM, and none for MAPPO.

At thirty-seven gNBs the lead grows further, in an extension pre-registered before any of its runs. The configuration scales the arrivals exactly as before, so each gNB still sees 1.15 pending requests per step, and no setting was tuned for this size. AXIOM runs 30 seeds on the hex layout and 15 on each of the other two. The three baselines never read the layout in Env A, so each runs once on the 30 hex seeds. AXIOM keeps URLLC and eMBB at 96.7% and 90.7%, a pooled priority-slice compliance of 93.7% with 95% CI [89.8%, 97.2%]. The single-agent DQN, MAPPO, and the independent DQN keep 58.7%, 56.7%, and 32.1%. Under every layout AXIOM leads every baseline by 32.0 to 67.8 percentage points of compliance and by 18.3 to 29.5 reward per step, and all 18 comparisons hold after Holm correction within their pre-registered families, at adjusted $p < 0.01$. AXIOM collapses in 6 of its 60 runs, more often than at nineteen gNBs, against 26 of 30 for the single-agent DQN. Even its collapsed runs keep 59% to 65%, the level of the best baselines. A replication on 20 fresh seeds, also pre-registered, confirms the result. There AXIOM keeps 95.1% priority-slice compliance against 58.5%, 57.4%, and 34.1% for MAPPO, the single-agent DQN, and the independent DQN, and it beats each of them on compliance and on reward at Holm-adjusted $p < 0.001$. It collapses in 1 of the 20 seeds, against 18 for the single-agent DQN. Section S15 of Online Resource 1 gives the setup and every arm's results.

Fig. 5a traces the same result across sizes. AXIOM's priority-slice compliance stays between 97.1% and 98.9% from three to nineteen gNBs while every baseline falls, so its lead over the best baseline grows to 32.0 points at nineteen gNBs, with a parameter count that does not change. At thirty-seven gNBs every baseline falls again and the lead reaches 35.0 points. The static rule that sheds every mMTC request keeps 99.96% to 99.99% at every size. Where that rule is known, AXIOM comes within 3 points of it up to nineteen gNBs and within 6.3 at thirty-seven, while from seven gNBs up every other learner falls at least 24 points short. Fig. 6 shows where that gap comes from, gNB by gNB, in the thirty-seven-gNB hex cluster. A gNB's admission gate for a slice is the PRB-ratio ceiling that its accept and reject decisions move between the slice's floor and cap, and we average its openness over every evaluation step of all 30 seeds. AXIOM keeps the URLLC and eMBB gates open at every gNB, at 1.00 and 0.98, while closing the mMTC gate to 0.16, a mean that includes its three collapsed seeds, so it sheds the right slice everywhere in the cluster. The single-agent DQN leaves every gate open at 0.93 to 0.95, the independent DQN closes the mMTC gate only to 0.94, and MAPPO partly closes the eMBB gate, to 0.80, while leaving mMTC open at 0.51. The nineteen-gNB cluster of the main campaign gives the same picture. There AXIOM keeps the two gates at 1.00 and 0.97, while closing the mMTC gate to 0.08. The single-agent DQN leaves every gate open at 0.99, the independent DQN closes the mMTC gate only to 0.92, and MAPPO partly closes the eMBB gate, to 0.82, while leaving mMTC open at 0.69.

Fig. 7a places the four learned policies on one map of accuracy at three gNBs against priority-slice protection at nineteen, in Env A. AXIOM sits highest on both axes, at 12.70 reward per step and 97.3% compliance. The single-agent DQN reaches 12.39 and 65.3%, the independent DQN 11.59 and 56.4%, and MAPPO 10.01 and 60.6%. AXIOM's reward is 98.9% of the oracle rule's 12.835. The radar in Fig. 7b adds Env B reward at three gNBs, collapse resistance, and Env B compliance at seven

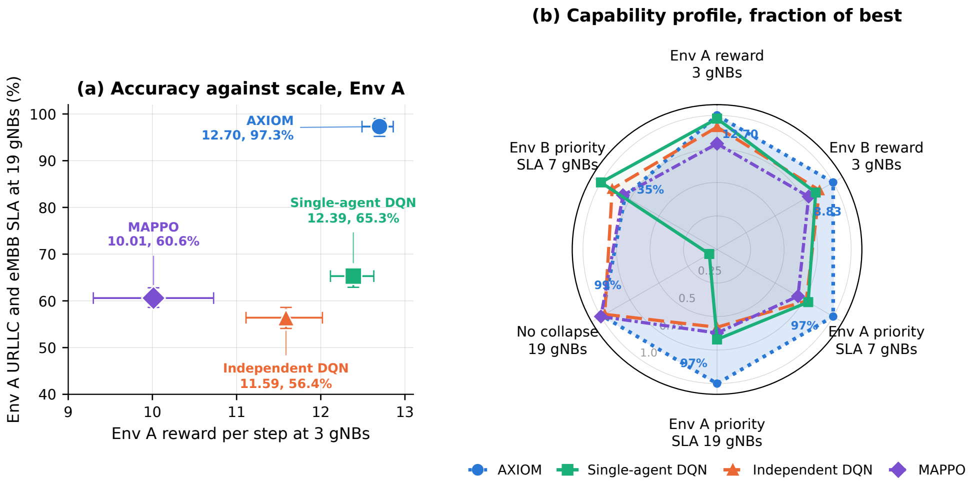

Fig. 7 AXIOM against the three learned baselines. a, Env A reward per step at three gNBs against compliance at 19 gNBs, with 95% CIs. b, Six measured axes, each as a fraction of the best learned policy on it

gNBs. AXIOM leads on five of the six axes shown. The exception is Env B compliance at seven gNBs, where the single-agent DQN leads.

5.3 Ablations and Sensitivity

Three checks test where AXIOM's behaviour comes from. First, the learning rate matters for AXIOM and its baselines alike. In Env B the original AXIOM, at 10^{-4}, earns 2.37 less per step than the current design in all ten seeds, while a faster rate or a wider encoder changes nothing significant, and Section S12 of Online Resource 1 gives the full study.

Second, graph attention adds nothing measurable anywhere we tested it. In Env A the per-node MLP variant ties AXIOM at three gNBs, with a difference of +0.058 and 95% CI [−0.17, 0.26], and at nineteen gNBs on the ring layout, +0.31 per step with 7 of 12 seeds tied. Env B's inter-cell coupling gives neighbour information real value, yet the pre-registered test H3 at seven gNBs even leans towards the MLP. The MLP keeps 1.12 and 0.95 more reward per step on the ring and hex layouts, winning 10 of 15 seeds in each, although neither difference survives Holm correction, at $p = 0.061$ and $p = 0.095$. AXIOM's gains therefore come from its shared encoder, per-slice heads, and joint training rather than from attention. One explanation is that a gNB's own queues already reflect most of what its neighbours do to it, so attending to them adds little.

Third, AXIOM's behaviour holds in Env A when the environment changes. Under three changed conditions, namely load at 1.02 and 1.07 times the nominal shares and the changed slice weights, AXIOM stays within 0.13 of the oracle rule's reward with no significant difference. On fresh seeds the gap at 1.02 times the load was larger in a ten-seed sample but not in a twenty-seed one, as Section 5.8 reports. On these seeds it keeps a positive lead over both DQN baselines under every condition, but at ten seeds only two of those six leads survive correction. They are its lead over the independent DQN at 1.07 times the load, +1.073 at $p = 0.006$, and over the single-agent DQN under the changed weights, +1.264 at $p = 0.041$.

Fourth, the per-slice Q-heads matter as the cluster grows. A variant with one shared head of matched size ties AXIOM at three gNBs in both environments. At nineteen gNBs on the ring layout, however, it collapses in 9 of 15 seeds and keeps only 65.9% priority-slice compliance against AXIOM's 96.8% on the same seeds, losing every seed at Holm-adjusted $p < 0.001$. Training on one TD sample per request, by contrast, is not what makes AXIOM work. A variant that instead regresses the sum of a step's chosen values on one joint target, as value decomposition does, is never significantly different and trends slightly better, reaching 12.831 reward per step in Env A at three gNBs and 99.96% priority-slice compliance at nineteen. Because this was found after the reported campaigns, we tested it once on 60 fresh seeds across four cells. It was again never significantly different from AXIOM, ahead in three cells and behind at seven gNBs in Env B, so we keep the reported design.

The comparison also holds when the baselines are made stronger. A grid of 24 MAPPO configurations, chosen on pilot seeds and allowing three times the training budget, kept the reported setting in Env B and selected a longer-trained one in Env A. There MAPPO rises from 10.013 to 11.448 reward per step at three gNBs and from 60.6% to 69.6% priority-slice compliance at nineteen gNBs on the ring layout. AXIOM still beats it in every cell tested, by 1.25 to 16.18 reward per step at Holm-adjusted $p < 0.001$, and by 2.42 in Env B at seven gNBs. At thirty-seven gNBs, in a check reported but not predicted, the selected MAPPO keeps 65.6% priority-slice compliance, and AXIOM beats it by 29.2 points and 23.23 reward per step on 15 seeds, at unadjusted $p < 0.001$. With its initialisation properly seeded, the single-agent DQN keeps 80.3% and 63.9% priority-slice compliance at seven and nineteen gNBs, and AXIOM beats it under every layout at Holm-adjusted $p \leq 0.035$.

5.4 Time-Varying Load with Inter-Cell Coupling

Env B tests the case a static benchmark cannot, where no fixed rule is optimal. Table 6 and Fig. 8 report the thirty-seed campaign at three gNBs, and fifteen seeds per layout

Table 6 Env B, three gNBs, 30 seeds per arm, 95% bootstrap CIs. Rules above the line, learned policies below it

Arm	Reward/step	URLLC	eMBB	mMTC
Accept all	4.356 [4.13, 4.60]	0.446	0.265	0.955
Shed all mMTC	6.337 [6.00, 6.67]	0.625	0.367	0.371
Tuned local rule	9.252 [9.12, 9.40]	0.989	0.367	0.390
Tuned neighbour-aware rule	9.041 [8.93, 9.15]	0.988	0.283	0.901
AXIOM	8.832 [8.64, 9.03]	0.958	0.344	0.406
AXIOM, MLP encoder	8.948 [8.77, 9.13]	0.974	0.337	0.452
Single-agent DQN	7.495 [7.20, 7.77]	0.866	0.284	0.412
Independent DQN	7.794 [7.44, 8.13]	0.910	0.270	0.441
MAPPO	6.968 [6.47, 7.45]	0.797	0.289	0.579

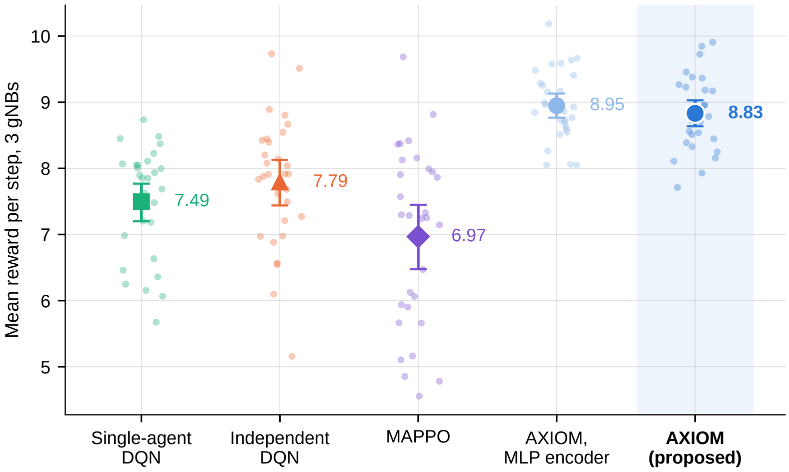

Fig. 8 Env B reward per step of the learned policies at three gNBs, 30 seeds, with per-seed points behind the mean and 95% CI

run at seven. First, adapting to the load pays off clearly. Shedding every mMTC request, the optimum of Env A, now leaves URLLC compliance at 62.5%. AXIOM beats that rule by +2.495 reward per step and accept-all by +4.476, winning all 30 seeds against each at Holm-adjusted $p < 0.0001$, as hypothesis H1 predicted. AXIOM keeps URLLC at 95.8% by shedding eMBB as well as mMTC when URLLC queues build.

Second, among learned policies AXIOM leads at three gNBs. It beats the independent DQN, the single-agent DQN, and MAPPO by +1.038, +1.337, and +1.863 reward per step and also on priority-slice compliance, winning 24 to 29 of 30 seeds, all at Holm-adjusted $p < 0.0001$. The disjoint ten-seed replication points the same way, with AXIOM winning 7 or 8 of 10 seeds against each, but no lead survives correction there.

Third, a tuned rule beats every learned policy. The best local rule sheds every mMTC request and sheds eMBB whenever the URLLC queue passes 0.2, and it earns 9.252 per step, 0.420 more than AXIOM, winning 22 of 30 seeds at $p < 0.0001$. The neighbour-aware rule earns slightly less, at 9.041, so neighbour information did not help the rules either. The rule encodes domain knowledge of which slice to shed under which pressure, which AXIOM must learn from the reward alone in 300 episodes. No policy we tested keeps eMBB compliance above 37% in Env B, because demand at the load peaks exceeds the budget by up to 23.5% and a slice counts as compliant only when every gNB meets it. On 15 seeds, even the rule with the highest eMBB compliance among all 95 rules of the local family, chosen in hindsight on each evaluation stream, reaches only 37.7% on average. The best rule by reward in hindsight earns only 0.14% more than the tuned rule.

At seven gNBs every learned policy falls further behind. The tuned rules earn 26.5 to 27.3 per step against 20.3 to 24.6 for the learned policies, and they beat AXIOM in all 15 seeds under every layout. AXIOM itself earns 23.4 to 23.6, no more than the static rule's 23.6, and keeps priority-slice compliance at 34% to 36%, close to the static rule. On the ring layout that is below the single-agent DQN, at Holm-adjusted $p = 0.011$. AXIOM's only significant leads over a learned baseline at this size are over MAPPO's reward, under the fully-connected and ring layouts. In Env B, learned multi-agent admission control therefore beats a static rule only at three gNBs, and never beats a rule tuned with domain knowledge under the conditions that rule was tuned for.

That lead depends on the conditions matching those the rule was tuned for. When the load rises after tuning and training, with neither the rule re-tuned nor AXIOM retrained, AXIOM overtakes the rule. At 1.10, 1.15, and 1.20 times the nominal load it leads the nominally tuned rule by 0.46, 0.43, and 0.35 reward per step on 25 checkpoints not used to find this effect, winning 21 to 23 of 25 seeds at Holm-adjusted $p \leq 0.0002$. At seven gNBs on the ring layout the reversal is larger, from a deficit of 3.84 under nominal load to leads of 1.86 and 3.14 at 1.10 and 1.20 times the load. A rule re-tuned on the shifted conditions draws level with AXIOM at 1.10 and 1.15 times the load and beats it by 0.62 at 1.20. Under lighter load or stronger coupling the tuned rule keeps a lead that is not significant.

Longer training also narrows the gap under nominal conditions. With three times the training budget AXIOM gains 0.33 reward per step at three gNBs and 2.92 at seven, on 20 and 12 seeds. The remaining gaps to the tuned rule, 0.08 and 0.96 per step, are no longer significant, at $p = 0.35$ and $p = 0.064$.

5.5 Federated Training and DP

With federation switched on and no noise, mean reward per step on the ten shared seeds is 12.356 for centralised AXIOM and 11.929 for the federated variant. That is a difference of +0.427 with 95% CI [−0.54, 1.51] and $p = 0.65$, so federation shows no significant cost on these seeds, although it is more variable, with five seeds losing and four gaining. The disjoint replication shows a larger cost of +0.968, with 95% CI [0.18, 1.96], which falls just short of significance at $p = 0.055$. Pooled over all 30 federated seeds, including ten fresh ones, the cost is +0.491 per step with 95% CI [−0.08, 1.09] and $p = 0.14$, so federation costs little on average, but a cost of up to about one reward per step cannot be excluded.

Adding DP-SGD noise to every step lowers reward by 0.77 to 1.25 per step, and no drop survives Holm correction at ten seeds. The formal guarantee is weak at these settings, with ε of about 2,600 or more, so per-step DP-SGD delivers no meaningful guarantee here, and Section S12 of Online Resource 1 reports the privacy analysis in full.

5.6 Disruption

We inject three mid-episode disruptions, gNB dropout, demand spikes, and agent churn, into every policy's frozen Env A checkpoint without retraining. Dropout and churn hurt every policy significantly at every severity, and no gap between AXIOM and any baseline survives Holm correction under any disruption. Demand spikes leave AXIOM's decisions largely intact, with block precision at 0.98, and Section S12 of

Online Resource 1 gives the protocol and every result. With ten seeds per arm these tests detect only large gaps, so smaller differences between policies remain possible.

5.7 Model Complexity

Table 7 sets out the cost of each policy at every cluster size. AXIOM's shared encoder keeps its parameter count at 7,782 at every cluster size, whereas the independent DQN grows from 55,302 to 682,058 parameters at thirty-seven gNBs and the single-agent DQN's input grows with the cluster. AXIOM's greedy decisions take 1.0 to 9.9 ms per control step on one CPU thread, well inside the 5 s control period of the live testbed. MAPPO is fastest at inference, because it scores all of a step's requests in one batch.

5.8 Reproducibility and Fresh-Seed Verification

We checked the results in three ways, with every pass criterion fixed before the run it judged. First, rerunning eleven reported cells on their original seeds in fresh processes reproduced every stored per-seed result exactly, with one exception. The single-agent DQN's training code builds its network before setting the seed, so the first seed of each batch starts from an unseeded initialisation. Rerunning this baseline with its initialisation seeded leaves AXIOM's leads unchanged, in Env B at three gNBs and under every Env A layout at seven and nineteen gNBs.

Second, four further runs retrained every arm on fresh seeds, 10 to 40 per cell, and 81 of their 90 pre-stated checks held. At seven and nineteen gNBs in Env A AXIOM kept 93.4% to 100% priority-slice compliance and beat every baseline under every layout, at Holm-adjusted $p \leq 0.032$ in all but one ten-seed cell, which a twenty-seed rerun then confirmed at $p \leq 0.007$. In Env B at three gNBs it again beat the static rule and every learned baseline at Holm-adjusted $p \leq 0.0002$, the tuned rules again beat every learned policy, and the disruption, federation, and privacy results held as reported. Online Resource 1 reports every number from these runs.

The nine checks that failed concern secondary margins rather than any headline result. On fresh seeds the DQN baselines reached up to 83.3% compliance at seven gNBs, above the 80% bound we had set. AXIOM's small lead over the single-agent DQN in Env A at three gNBs did not recur, at -0.021 and $+0.159$ per step on 20 and 40 new seeds, and at 1.02 times the load that baseline matched AXIOM. Two weak AXIOM seeds also left one ten-seed load sample 3.2% short of the oracle, a gap that a twenty-seed rerun did not reproduce. Third, a script written independently of the

Table 7 Model complexity at 3, 7, 19, and 37 gNBs. Parameters used at execution, greedy inference time per control step on one CPU thread, and median training time per seed at 3, 19, and 37 gNBs, on the ring layout for AXIOM

Policy	Parameters	ms per step	Training s
AXIOM	7,782 at every size	1.04 / 1.98 / 4.79 / 9.89	624 / 892 / 1182
AXIOM, MLP encoder	3,190 at every size	0.77 / 1.69 / 4.46 / 9.44	503 / – / –
Single-agent DQN	21,250 / 26,370 / 41,730 / 64,770	0.61 / 1.42 / 3.95 / 8.02	85 / 202 / 358
Independent DQN	55,302 / 129,038 / 350,246 / 682,058	0.59 / 1.38 / 3.73 / 8.08	181 / 1054 / 2088
MAPPO	5,122 at every size	0.19 / 0.20 / 0.22 / 0.27	73 / 256 / 495

analysis pipeline, with its own log reader and Holm correction, recomputes from the raw logs the numbers in every results table and the key results sentences, and all 437 of its checks pass.

5.9 Live Deployment

We also checked the deployment path that AXIOM would use on a real single-gNB OAI testbed, running an offline-trained single-agent DQN checkpoint through the real E2 control loop with real UEs and per-slice User Datagram Protocol (UDP) traffic. The pipeline worked end to end, with 0.000% loss at both loads and zero below-floor grants across more than 100,000 readings after a scheduler-configuration fix. Because the rig never ran short of capacity at either load, this check validates the control path but cannot show admission behaviour under contention. AXIOM itself is evaluated offline, since our two-gNB live attempt did not sustain traffic, and Section S9 of Online Resource 1 gives the testbed and every live result.

6 Conclusion

This paper presented AXIOM, a multi-agent slice admission controller with a shared cluster encoder and decentralised per-slice action heads, together with an evaluation protocol designed to make its results trustworthy. It targets three gaps from our prior systematic review. It closes the first, trustworthy evaluation, tests the second, privacy-preserving coordination, and finds it still open, and validates only the control path of the third, live deployment.

AXIOM's clearest advantage appears as the cluster grows. In a static-load shared-budget environment it keeps priority-slice compliance above 93% from seven to thirty-seven gNBs, while single-agent and independent DQNs and MAPPO stay below 84% and only a static rule known in advance does better. Its parameter count stays at 7,782 at any size, and it reaches 98.9% of an oracle rule's reward at three gNBs. In a harder environment with time-varying load and inter-cell coupling, AXIOM beats the static and accept-all rules and leads every learned baseline at three gNBs, and federated training showed no significant cost, although a moderate one cannot be excluded. These gains have two limits that we state plainly. A three-parameter rule tuned with domain knowledge still beats every learned policy under coupling at the conditions it was tuned for, although AXIOM overtakes it when the load rises beyond them and nearly closes the gap with three times the training. The single-agent DQN keeps pace with AXIOM in Env A at three gNBs and in Env B at seven. Graph attention added nothing over a per-node encoder, and ablations show that the per-slice heads carry AXIOM's advantage at scale, and per-step DP-SGD gave only a weak guarantee. The main findings held on fresh seeds, and the live testbed validates the control path rather than AXIOM itself, which is evaluated offline.

The evaluation safeguards changed what this paper finally reports, exposing an inert earlier simulator, queue backlog carried over between episodes, and an AXIOM design that learned only a static rule, and every reported result comes from the corrected versions. The broader lesson is that learned admission control should be judged against trivial and tuned rules, with metrics tied to the objective, on independent seeds, under a pre-registered analysis.

Future work targets the gaps that this paper leaves open. The first is a live multi-gNB deployment of AXIOM itself. The second is closing the remaining gap to tuned rules in coupled environments, which three times the training already narrows, for instance by giving the learner the queue-threshold structure the rules exploit. The third is a privacy layer that adds noise to the per-round model updates rather than to every step, so that the formal guarantee becomes meaningful. We also plan Multi-Access Edge Computing (MEC) scaling, which adds compute-placement nodes to the cluster graph, cross-operator federation, and direct Quality of Experience (QoE) inference in the reward.

Supplementary information. Online Resource 1 is a PDF that gives the full verification results, the live testbed check, additional setup details, and additional results.

Acknowledgements. This work was supported by the Fundamental Research Grant Scheme (FRGS) of the Ministry of Higher Education (MOHE), Malaysia, under grant FRGS/1/2024/ICT06/TAYLOR/02/1.

Statements and Declarations

The following statements should be understood to apply to this manuscript unless otherwise indicated.

- **Funding:** This work was supported by the Fundamental Research Grant Scheme (FRGS), Ministry of Higher Education (MOHE), Malaysia, under grant number FRGS/1/2024/ICT06/TAYLOR/02/1.
- **Competing interests:** The authors declare no competing interests.
- **Ethics approval and consent to participate:** Not applicable, since this study involves no human participants, animal subjects, or identifiable patient data. All reported results are either offline simulation campaigns or measurements from a laboratory radio testbed.
- **Consent for publication:** Not applicable, since this manuscript contains no personal data or identifying information about any individual.
- **Data availability:** The experiment logs, checkpoints, and campaign result files underlying this paper's findings are available from the corresponding author on reasonable request.
- **Materials availability:** Not applicable, since this study involves no physical or biological materials.
- **Code availability:** The framework and experiment scripts used to produce this paper's results, including the reproducibility manifest and analysis scripts used to independently re-derive every table and figure in this paper, are available from the corresponding author on reasonable request.
- **Author contributions:** M. P. Kunasegran: conceptualisation, methodology, software, investigation, writing (original draft). W. L. Pang: supervision, writing (review and editing). S. K. Phang: supervision, writing (review and editing).

References

[1] Abadi M, Chu A, Goodfellow I, et al (2016) Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp 308–318, https://doi.org/10.1145/2976749.2978318

[2] Ahmadi M, Moayyedi A, Sulaiman M, et al (2024) Generalizable 5G RAN/MEC slicing and admission control for reliable network operation. IEEE Trans Netw Service Manag https://doi.org/10.1109/TNSM.2024.3437217

[3] AlQahtani SA (2025) Dynamic slice allocation in 5G cloud RAN. Wireless Personal Communications 143(3-4):427–467. https://doi.org/10.1007/s11277-025-11835-y

[4] Amiri E, Wang N, Shojafar M, et al (2024) Edge-AI empowered dynamic VNF splitting in O-RAN slicing: A federated DRL approach. IEEE Communications Letters 28(2):318–322. https://doi.org/10.1109/LCOMM.2023.3341302

[5] Bega D, Gramaglia M, Banchs A, et al (2020) A machine learning approach to 5G infrastructure market optimization. IEEE Transactions on Mobile Computing 19(3):498–512. https://doi.org/10.1109/TMC.2019.2896950

[6] Bouroudi A, Outtagarts A, Hadjadj-Aoul Y (2025) GNN-based multi-agent DRL for energy-efficient multi-domain 6G resource orchestration. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 4457–4462, https://doi.org/10.1109/GLOBECOM59602.2025.11432814

[7] Bun M, Steinke T (2016) Concentrated differential privacy: Simplifications, extensions, and lower bounds. In: Theory of Cryptography Conference (TCC), pp 635–658, https://doi.org/10.1007/978-3-662-53641-4_24

[8] Doanis P, Spyropoulos T (2024) Sample-efficient multi-agent DQNs for scalable multi-domain 5G+ inter-slice orchestration. IEEE Transactions on Machine Learning in Communications and Networking 2:956–977. https://doi.org/10.1109/TMLCN.2024.3420268

[9] Fatehi K, Rahmani Ghourtani M, Sonee A, et al (2026) Interpretable attention-based multi-agent PPO for latency spike resolution in 6G RAN slicing. In: Proc. IEEE Int. Conf. Commun. (ICC), https://doi.org/10.1109/ICC59461.2026.11586913

[10] Gong Y, Gong P, Jiang H, et al (2026) Intelligent resource allocation for multiaccess edge computing in 5G ultra-dense slicing network using federated multiagent DDPG. China Commun 23(1):273–289. https://doi.org/10.23919/JCC.fa.2022-0187.202601

[11] Hao X, She C, Yeoh PL, et al (2024) Hybrid-task meta-learning: A GNN approach for scalable and transferable bandwidth allocation. IEEE Transactions on Wireless Communications 23(12):19820–19835. https://doi.org/10.1109/TWC.2024.3487533

[12] Kamal YB, Abdulkafi AA (2026) Reinforcement learning double deep Q-network techniques for NextG network. Wireless Personal Communications 146(6):3175–3197. https://doi.org/10.1007/s11277-026-12048-7

[13] Khani M, Jamali S, Sohrabi MK, et al (2024) Slice admission control in 5G cloud radio access network using deep reinforcement learning: A survey. International Journal of Communication Systems 37(13):e5857. https://doi.org/10.1002/dac.5857

[14] Krishnamoorthy R, Gireesh Babu CN, Gite P, et al (2026) Federated learning for dynamic resource allocation in 6G network slicing. Wireless Personal Communications https://doi.org/10.1007/s11277-026-11943-3

[15] Kulkarni D, Venkatesan M, Kulkarni AV (2024) Actor critic based reinforcement learning for joint resource allocation and throughput maximization in 5G RAN slicing. Wireless Personal Communications 138(1):623–640. https://doi.org/10.1007/s11277-024-11526-0

[16] Kunasegran MP, Pang WL, Phang SK (2025) Optimising resource allocation in 5G networks: Balancing URLLC and eMBB traffic under gNB congestion. In: 2025 Multimedia University Engineering Conference (MECON), pp 1–6, https://doi.org/10.1109/MECON67253.2025.11277096

[17] Kunasegran MP, Pang WL, Phang SK (2025) SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN. In: 2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE), pp 207–212, https://doi.org/10.1109/ICRAIE65839.2025.11239148

[18] Kunasegran MP, Pang WL, Phang SK (2026) Comprehensive review of optimization techniques for user-centric distributed network slicing in 5G networks. IEEE Access 14:114630–114648. https://doi.org/10.1109/ACCESS.2026.3716911

[19] Li T, Sahu AK, Talwalkar A, et al (2020) Federated learning: Challenges, methods, and future directions. IEEE Signal Processing Magazine 37(3):50–60. https://doi.org/10.1109/MSP.2020.2975749

[20] Li Y, Wang H, Min G, et al (2026) A flexible and scalable multi-agent learning framework for dynamic RAN slicing in 6G native-AI networks. IEEE Transactions on Mobile Computing 25(5):7258–7273. https://doi.org/10.1109/TMC.2025.3642213

[21] Lim WYB, Luong NC, Hoang DT, et al (2020) Federated learning in mobile edge networks: A comprehensive survey. IEEE Communications Surveys & Tutorials 22(3):2031–2063. https://doi.org/10.1109/COMST.2020.2986024

[22] Ming Z, Yu H, Taleb T (2024) Federated deep reinforcement learning for prediction-based network slice mobility in 6G mobile networks. IEEE Transactions on Mobile Computing 23(12):11937–11953. https://doi.org/10.1109/TMC.2024.3404125

[23] Mohajer A, Mirzaei A, Darabi M, et al (2026) Joint SLA-aware task offloading and adaptive service orchestration with graph-attentive multi-agent reinforcement learning. IEEE Trans Netw Service Manag 23:3737–3754. https://doi.org/10.1109/TNSM.2026.3673188

[24] Ojijo MO, Ramotsoela D, Oginga RA (2024) Slice admission control in 5G wireless communication with multi-dimensional state space and distributed action space: A sequential twin actor-critic approach. Computer Networks 255:110878. https://doi.org/10.1016/j.comnet.2024.110878

[25] Phyu HP, Naboulsi D, Stanica R (2025) ICE-CREAM: Multi-agent fully cooperative decentralized framework for energy efficiency in RAN slicing. IEEE Transactions on Network and Service Management 22(2):1859–1873. https://doi.org/10.1109/TNSM.2024.3524503

[26] Polese M, Bonati L, D'Oro S, et al (2023) ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms. IEEE Transactions on Mobile Computing 22(10):5787–5800. https://doi.org/10.1109/TMC.2022.3188013

[27] Polese M, Bonati L, D'Oro S, et al (2023) Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. IEEE Communications Surveys & Tutorials 25(2):1376–1411. https://doi.org/10.1109/COMST.2023.3239220

[28] Qasim M, Imran M, Akram F, et al (2025) Transformer and graph neural network-based federated learning for cellular traffic prediction with sustainability analysis. IEEE Access 13:159649–159666. https://doi.org/10.1109/ACCESS.2025.3605380

[29] Rashid T, Samvelyan M, Schroeder de Witt C, et al (2020) Monotonic value function factorisation for deep multi-agent reinforcement learning. Journal of Machine Learning Research 21(178):1–51

[30] Raza MR, Natalino C, Öhlen P, et al (2019) Reinforcement learning for slicing in a 5G flexible RAN. Journal of Lightwave Technology 37(20):5161–5169. https://doi.org/10.1109/JLT.2019.2924345

[31] Shao Y, Li R, Hu B, et al (2021) Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network. IEEE Trans Veh Technol 70(10):10792–10803. https://doi.org/10.1109/TVT.2021.3103416

[32] Sirma M, Kavak A, Inner AB (2025) Optimizing relay selection in D2D communication for next-generation wireless networks using multi-agent reinforcement learning: A novel approach. Wireless Personal Communications 140(3-4):945–969. https://doi.org/10.1007/s11277-025-11753-z

[33] Sulaiman M, Moayyedi A, Ahmadi M, et al (2023) Coordinated slicing and admission control using multi-agent deep reinforcement learning. IEEE Trans Netw Service Manag 20(2):1110–1124. https://doi.org/10.1109/TNSM.2022.3222589

[34] Tao Z, Xu W, You X (2025) Digital twin-accelerated online deep reinforcement learning for admission control in sliced communication networks. IEEE Transactions on Communications 73(4):2490–2504. https://doi.org/10.1109/TCOMM.2024.3476430

[35] Tashman DH, Cherkaoui S (2026) Adversarial attacks in AI-driven RAN slicing: SLA violations and recovery. In: Proc. Int. Wireless Commun. and Mobile Computing (IWCMC), https://doi.org/10.1109/IWCMC69287.2026.11580033

[36] Tsampazi M, D'Oro S, Polese M, et al (2023) A comparative analysis of deep reinforcement learning-based xApps in O-RAN. In: Proc. IEEE Global Communications Conference (GLOBECOM), pp 1638–1643, https://doi.org/10.1109/GLOBECOM54140.2023.10437367

[37] Tsampazi M, D'Oro S, Polese M, et al (2025) PandORA: Automated design and comprehensive evaluation of deep reinforcement learning agents for open RAN. IEEE Transactions on Mobile Computing 24(4):3223–3240. https://doi.org/10.1109/TMC.2024.3505781

[38] Wu Z, Pan S, Chen F, et al (2021) A comprehensive survey on graph neural networks. IEEE Transactions on Neural Networks and Learning Systems 32(1):4–24. https://doi.org/10.1109/TNNLS.2020.2978386

[39] Yasin TV, Yu CM, Wang LC (2025) Differential privacy federated edge learning-assisted for securing RAN intelligent controller in O-RAN 6G communications. In: Proc. IEEE VTS Asia Pacific Wireless Commun. Symp. (APWCS), https://doi.org/10.1109/APWCS67981.2025.11151868

[40] Yu C, Velu A, Vinitsky E, et al (2022) The surprising effectiveness of PPO in cooperative multi-agent games. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, pp 24611–24624, https://doi.org/10.52202/068431-1787

[41] Zangooei M, Golkarifard M, Rouili M, et al (2024) Flexible RAN slicing in Open RAN with constrained multi-agent reinforcement learning. IEEE Journal on Selected Areas in Communications 42(2):280–294. https://doi.org/10.1109/JSAC.2023.3336156

[42] Zhang H, Zhou H, Erol-Kantarci M (2022) Federated deep reinforcement learning for resource allocation in O-RAN slicing. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 958–963, https://doi.org/10.1109/GLOBECOM48099.2022.10001658